

clase 06/10 Testing

🕒 Creadas	6 de octubre de 2026 11:46
👤 green sw	ISW

Testing

Proceso destructivo de tratar encontrar defectos en el código (se asume su presencia)

se debe ir con una actitud negativa para demostrar que algo es incorrecto

Aprox. el 30 a 50% del costo de un software confiable

testing → encontrar defectos en un sistema/producto/servicio

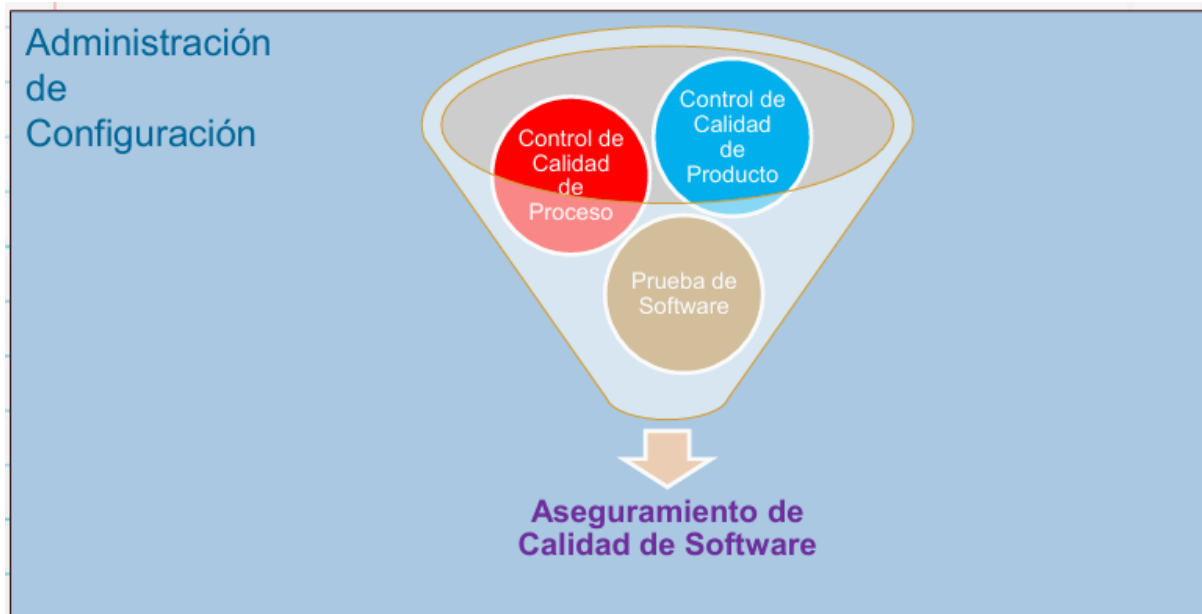
cuando terminas el testing, NO puedes asegurar que el programa este impoluto

calidad

Adecuación de un determinado producto o servicio a las necesidades del cliente que lo pide, Totalmente subjetiva.

Nivel de cumplimiento a las necesidades y expectativas (no se asumen como "obvias").

no tiene posibilidad de garantizar ausencia de defectos, calidad y que cubra todo. El testing puede encontrar defectos, pero no asegura que sean todos.



la **Administración de Configuración (SCM)** es la base sobre la que se apoyan todas las actividades de calidad: por eso es el fondo que contiene el embudo.

Arriba de esa base están el control de calidad del proceso, el control de calidad del producto y la prueba de software, y todo eso junto "desemboca" en el **Aseguramiento de Calidad de Software (SQA)**.

Haces testing cuando Tienes una versión del producto construida.

Un testing exitoso es aquel que encuentra defectos.

Si haces un mal desarrollo, lleva fácilmente a un testing exitoso.

Un desarrollo bien hecho, lleva a un testing no exitoso.

PUD

Requerimientos

verificación → ¿estamos construyendo el producto correctamente? - se fija si el producto está construido correctamente según los requerimientos / la especificación

validación → ¿estamos construyendo el producto correcto? - la mide el cliente para ver si es lo que el quiere

Análisis

Diseño

Implementación

Acá se hacen las pruebas unitarias y las de integración (integración se podría definir entre esta y prueba en el caso que uses prácticas de CI)

Acá está el ambiente de desarrollo

Prueba

Acá están las pruebas de sistema

el testing llega tarde, porque se prueba lo que se construyó antes, entonces si o si llegan defectos no errores.

por eso el testing no aumenta la calidad, solo la mide/controla; la calidad se construye desde requerimientos.

Acá esta el ambiente de testing

Despliegue

Acá están las UAT (pruebas de aceptación)

Acá esta el ambiente de despliegue (preproducción y producción)

error y defecto no son sinónimos

error → queda en el contexto de la misma fase donde apareció

defecto → si no se encontró en la fase donde apareció, y se detectó en otra, es un defecto

un error de requerimientos, que no se corrigió en la fase de requerimientos va a salir más caro que corregirlo en la misma fase

falla → mal funcionamiento del sistema

no todos los defectos provocan fallas en el sistema (ej: defectos de diseño de interfaz [colores, formato, etc])

Testing es parte de asegurar calidad pero no es lo mismo

Una vez que se definen los requerimientos de calidad tengo que tener en cuenta:

- la calidad no puede inyectarse al final
- la calidad del producto depende de las tareas realizadas en el proceso
- detectar errores tempranamente, ahorra esfuerzo, tiempos y recursos
- La calidad no solamente abarca aspectos del producto sino también del proceso y como éstos se pueden mejorar, que a su vez evita defectos recurrentes.

Defectos, severidad y prioridad

¿Todos los defectos son iguales? NO

Cada equipo define sus niveles de severidad.

severidad → cuan grande o cuanto me afecta ese defecto

tiene que haber una definición escrita de que entendemos por cada nivel de severidad

Severidad	Prioridad
1 – Bloqueante	1 – Urgencia
2 – Crítico	2 – Alta
3 – Mayor	3 – Media
4 – Menor	4 – Baja
5 - Cosmético	

SEVERIDAD

Bloqueante → No se puede utilizar el *sistema*

Crítico → No puedo cumplir mis objetivos con esta *funcionalidad*, no tengo plan B

Mayor → Acá la funcionalidad no anda, pero *tengo un plan B* (ej: *querés ver un movimiento bancario, y no lo vez, pero la plata está ahí*)

Menor → Algo que hay que resolver, pero *no impide el funcionamiento* de la funcionalidad (ej: *no anda un filtro*)

Cosmético → Errores de ortografía, usabilidad, posiciones

PRIORIDAD

Tiene que ver con la urgencia del negocio

Lo asigna el P.O.

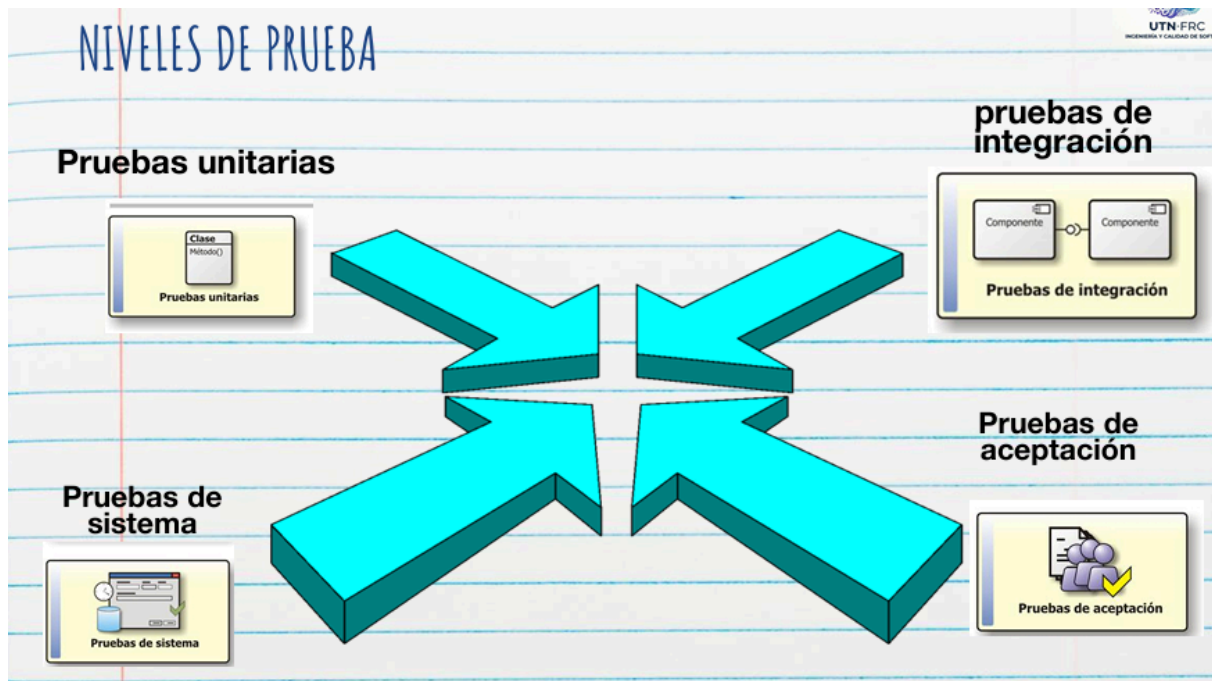
Urgencia → No puedo esperar nada, tiene que solucionarse ya (Se cayo el sistema de cobros en un supermercado)

De ahí, las demás prioridades se basan en cuanta es la tolerancia de espera a su solución

Usualmente el Bloqueante se asocia con Urgencia (suele ser así, pero no siempre)

La **severidad** mide el impacto técnico del defecto en el sistema, y la define normalmente quien testea.

La **prioridad** indica la urgencia con la que hay que resolverlo según el negocio, y la decide el cliente / PO / líder.



4 niveles de pruebas

pruebas unitarias

son las pruebas que si o si tiene que hacer el *desarrollador*. Son las pruebas que están integradas en el TDD (Test Driven Development)
Están en la fase de implementación

pruebas de integración (pruebas de interfaces)

Si un componente en las pruebas unitarias anduvo bien, y otro también, Si el segundo depende del primero, en conjunto deberían andar los dos
Si se trabaja con CI/CD, se automatizan las pruebas , entonces se ubicarían entre implementación y prueba. Los libros lo ponen enteramente en la implementación.

Pruebas de sistema

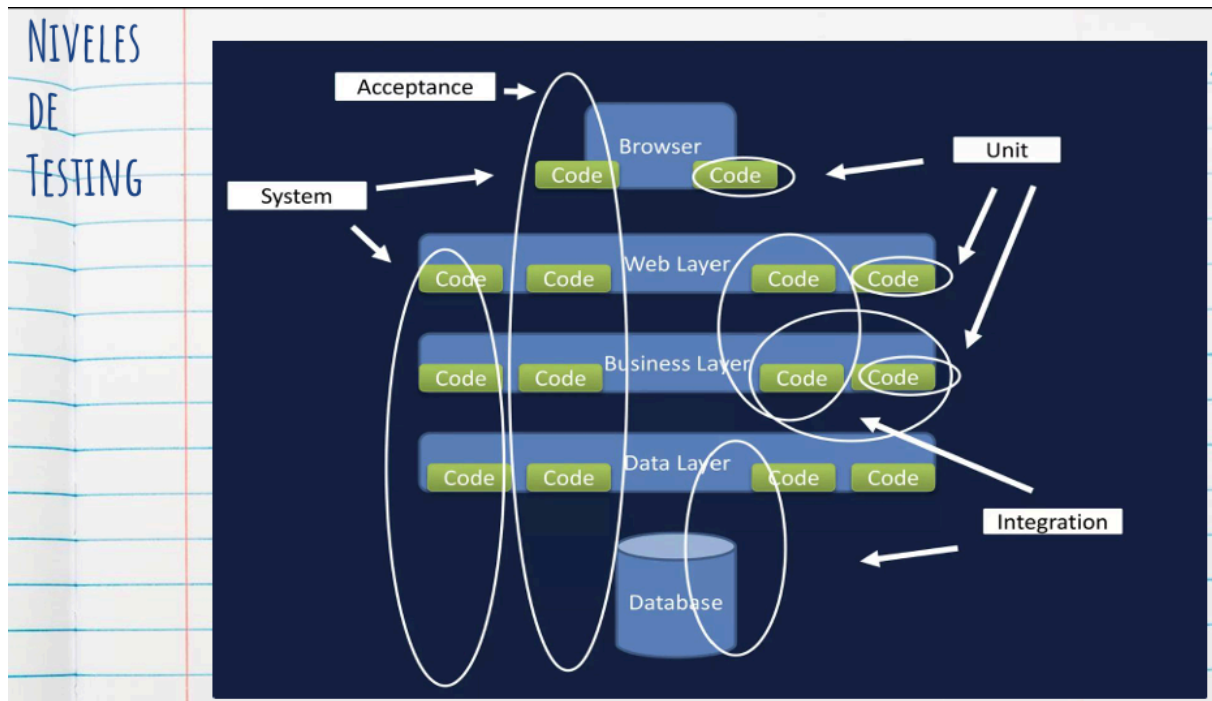
pertenecen al workflow de prueba.

Testea una porción vertical del sistema en completitud.

se arma el área de testing, con gente que trabaja en testing y tienen un jefe distinto al de que tienen los de software. Porque si fuese el mismo podría haber presiones que perjudicarían al sistema (por ej que se lance una implementación que no este lista para lanzarse)

Pruebas de aceptación (UserAcceptanceTests)

Las realiza el usuario con el equipo presente
Pertenece al workflow de despliegue



Cada componente de código que se prueba individualmente es una prueba unitaria

la integración junta distintos componentes dependientes

las pruebas de aceptación y de sistema son integradoras porque prueban todo el sistema

Se dice que son verticales, porque agarran porciones de todas las capas.

AMBIENTES PARA CONSTRUCCIÓN DEL SOFTWARE

Ambientes

Desarrollo

Prueba

Pre –
Producción

Producción

Configuración del espacio de trabajo con el repositorio, que vamos a usar todo el tiempo, con todas las herramientas necesarias para poder desarrollar y probar

Ambiente de desarrollo → es el conjunto de hardware y software (IDE, compiladores, librerías, repositorio, etc.) que usan los desarrolladores para construir el producto. Pertenece al workflow de implementación y **acá se hacen las pruebas unitarias** (y generalmente las de integración).

Ambiente de testing → es un entorno separado y controlado que usan los testers para probar el producto, y **los desarrolladores no deben tener acceso** a él, para garantizar la objetividad de las pruebas. Pertenece al workflow de prueba y **acá se hacen las pruebas de sistema**.

Ambiente de desarrollo ≠ ambiente de testing

no pueden trabajar en el mismo entorno la gente que esta desarrollando y la gente que esta probando

Se separan por la objetividad que viene a dar la gente que hace testing. Un area donde los desarrolladores no tengan injerencia, y lo mismo al revez.

Separar bien las áreas nos ayuda a solucionar el dilema de "En mi maquina andaba"

El Ambiente de testing esta en el workflow de prueba, y el de desarrollo en el de implementación

Ambiente de despliegue

El Ambiente de despliegue debería ser 2: preproducción y producción se encuentran en el workflow de despliegue

Ambiente de Preproducción → debe ser una réplica lo más fiel posible del ambiente de producción (mismo hardware, software, configuraciones y datos similares a los reales), para comprobar que el producto va a funcionar correctamente una vez desplegado. **Acá se hacen las pruebas de aceptación** con el cliente/usuario. Su problema es que replicar producción puede ser muy costoso o directamente imposible (por ejemplo, la infraestructura de servidores de Google)

Ambiente de Producción → es el ambiente real donde los usuarios finales usan el sistema para sus actividades. **Acá no se prueba**, porque cualquier falla tiene consecuencias directas sobre el negocio y los usuarios. Lo que llega a producción ya debería haber pasado todas las pruebas y cumplir con la DoD

Disclaimer: ¡¡¡Definiciones de ambientes BY CLAUDE!!!

Caso de prueba

Set de condiciones o variables bajo las cuales un tester determinara si el software esta funcionando correctamente o no

- Buena definición de casos de prueba nos ayuda a REPRODUCIR defectos
- la prueba de usuario que hacíamos para el P1, es el título con el que arrancamos a desmembrar para los casos de prueba.
- El paso a paso del testing tiene que ser específico
- El caso de prueba es el artefacto del workflow de testing
- No hagas testing sin casos de prueba

CASOS DE PRUEBA

**“Los bugs se esconden en las esquinas
y se congregan en los
límites ...”**

Boris Beizer



los defectos aparecen con mucha más frecuencia en los valores extremos que en los valores "normales"

objetivo → Encontrar errores

criterio → en forma completa

restricción → con el mínimo esfuerzo y tiempo

El testing exhaustivo es probar todos los caminos distintos

Podes llegar a tener 240.000 casos de prueba en un sistema chico, que si haces 1 prueba por segundo serian 8 dias, si fuese 1 min seria un año, si fuesen 10mins, 10 años.

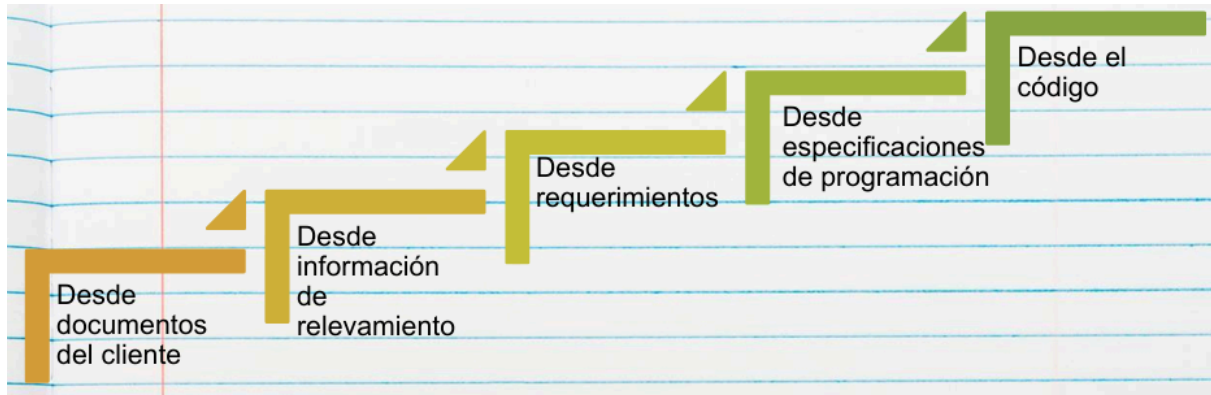
Por lo tanto no debería ser exhaustivo

Testing de caja negra y testing de caja blanca

Caja negra → se prueba el software **sin mirar el código**, solo desde afuera. Se ingresan datos y se compara la salida obtenida con la esperada según los requerimientos. Interesa **qué hace** el sistema, no cómo lo hace. Técnicas típicas: partición de equivalencias y análisis de valores límite.

Caja blanca → se prueba **conociendo y mirando el código** por dentro. Los casos de prueba se diseñan a partir de la estructura interna (camino, condiciones, decisiones, ciclos), buscando recorrer la mayor parte posible del código. Interesa **cómo lo hace**. Técnicas típicas: cobertura de sentencias, de decisiones y de condiciones.

¿De donde saco la informacion para los casos de prueba?



Si tenemos que diseñar casos de prueba y no tenemos escritos los requerimientos, no podemos hacer validación, pero verificación si.

El **proceso estándar** para testing sigue estos pasos:

1. **Planificación de pruebas** -sale → **Plan de pruebas**
2. **Diseño de caso de pruebas** -sale → **Caso de pruebas, BD para probar**
3. **Ejecución** -Entra → **Versión de sw y la salida del diseño** y -sale → **resultados de ejecución y defectos detectados**
4. **Reporte de defectos** -sale → **reporte de defectos, corrige, genera nueva version** y vuelve al paso 3
5. **FIN** (cuando se cumple el criterio de aceptacion)

Deberia haber optimamente 2 ciclos

en el ciclo 0 se encuentran defectos, se corrige y se manda

en el ciclo 1 no se encuentran defectos

cierro testing

Regresion

Al concluir un ciclo de pruebas y reemplazarse la versión del sistema, debe realizarse una verificación total de la nueva versión (volver a ejecutar todos los casos de prueba, no solo los que fallaron), para **detectar** nuevos defectos que se hayan introducido al corregir los anteriores.

cada defecto que se soluciona, introduce 3 mas

QUEDAMOS EN FILMINA 27